

```

import os
import pandas as pd
import time
import math
from sage.all import QQ, EllipticCurve, prod
from sage.schemes.elliptic_curves.ec_database import elliptic_curves
from concurrent.futures import ProcessPoolExecutor, as_completed

#
=====
=====
# SECTION 1: LOCAL DATABASE SETUP
#
=====
=====

os.environ["CREMONA_DATABASE_PATH"] = os.path.expanduser("~/ecdata")
os.environ["LMFDB_DATABASE_PATH"] = os.path.expanduser("~/lmfdb")

print(" Using local ecdata at:", os.environ.get("CREMONA_DATABASE_PATH"))
print("Using local lmfdb data at:", os.environ.get("LMFDB_DATABASE_PATH"))

DATA_DRIVEN_KAPPA = 31.5926
KAPPA = 1000.0
SQRT_KAPPA = math.sqrt(KAPPA)
VIRGO_DISTANCE = 5.4e7

def get_expanded_cluster_data():
    return {
        'Virgo':    {'r': 54, 'rho': 6320},
        'Coma':     {'r': 321, 'rho': 9980},
        'Perseus':  {'r': 236, 'rho': 11500},
        'Centaurus': {'r': 170, 'rho': 7500},
        'Fornax':   {'r': 62, 'rho': 3200},
        'Hercules': {'r': 500, 'rho': 8500},
        'Shapley':  {'r': 650, 'rho': 18000},
        'Horologium': {'r': 700, 'rho': 12000},
    }

#
=====
=====
# SECTION 2: WORKER FUNCTION FOR PARALLEL PROCESSING

```

```

#
=====

def process_cluster(args):
    """Worker function for parallel execution (must be top-level)."""
    name, data = args
    a, b = derive_curve_parameters(name, data['r'], data['rho'])

    print(f" > Processing '{name}' (a={a}, b={b})...")
    sage_info = create_minimal_model(a, b)
    projection = acsc_entropy_projection_hook(sage_info, data['r'], data['rho'])

    return {
        'Cluster': name,
        'r (Mly)': data['r'],
        'rho': data['rho'],
        'Derived a': a,
        'Derived b': b,
        'Minimal ainvs': str(sage_info.get('minimal_ainvs', 'N/A')),
        'Conductor': sage_info.get('conductor'),
        'Rank': sage_info.get('rank'),
        'Local Label': sage_info.get('local_label', '---'),
        '**projection',
        'Status': sage_info.get('status', 'Error')
    }

def derive_curve_parameters(cluster_name, r, rho):
    if cluster_name == 'Virgo':
        return -1706, 6320
    else:
        return round(-DATA_DRIVEN_KAPPA * r), rho

def create_minimal_model(a, b):
    print(f"  Creating Sage model for a={a}, b={b}...")
    try:
        E = EllipticCurve([0, a, 0, b, 0], minimal_twist=True)
        E_min = E.minimal_model()
        ainvs = E_min.ainvs()
        print(f"  Minimal model: {ainvs}")

        # === HIGH two_descent + ROBUST RANK ===

```

```

try:
    print(f"    Computing rank with high two_descent limit...")
    E_min.two_descent(second_limit=10000)
    rank = E_min.rank(only_use_mwrank=False)
except Exception:
    try:
        rank = E_min.rank_bound()
    except:
        rank = 0

# Local DB label lookup
label = "Unknown"
try:
    db = elliptic_curves
    cond = E_min.conductor()
    for curve in db.iter(curve_class=cond):
        if tuple(curve.ainvs()) == tuple(ainvs):
            label = getattr(curve, 'label', lambda: str(curve))()
            break
except:
    pass

return {
    'E': E_min,
    'minimal_ainvs': ainvs,
    'conductor': int(E_min.conductor()),
    'rank': rank,
    'discriminant': int(E_min.discriminant()),
    'real_period': float(E_min.period_lattice().real_period(prec=50)),
    'regulator': float(E_min.regulator()) if rank > 0 else 1.0,
    'tamagawa': float(prod(E_min.tamagawa_numbers())),
    'torsion_order': E_min.torsion_subgroup().order(),
    'local_label': label,
    'status': 'Success'
}
except Exception as e:
    print(f"    Sage Error: {str(e)}")
    return {'status': f'Sage Error: {str(e)}'}

```

#

```

=====
=====

```

SECTION 3: FULL ACSC + ENTROPY COHOMOLOGY PROJECTION

```

#
=====

def acsc_entropy_projection_hook(sage_info, cluster_r, cluster_rho):
    if sage_info.get('status') != 'Success':
        return {
            'projected_omega': None, 'h_eff_factor': None, 'comoving_volume': None,
            'scaled_regulator': None, 'entropy_proxy': None, 'cohomology_class': None,
            'cosmo_scale': None, 'note': sage_info.get('status')
        }

    rank = sage_info['rank']
    omega = sage_info['real_period']
    reg = sage_info['regulator']
    disc = abs(sage_info['discriminant'])

    cosmo_scale = VIRGO_DISTANCE / (omega * SQRT_KAPPA)
    scaled_period = omega * SQRT_KAPPA * cosmo_scale

    rank_divisors = {3: (1.5e13, 20), 2: (1.5e14, 7), 1: (8e13, 5)}
    volume_divisor, regulator_factor = rank_divisors.get(rank, (1e14, 20))

    comoving_volume = (omega * reg * (cosmo_scale ** 3)) / volume_divisor
    scaled_reg = reg * SQRT_KAPPA * regulator_factor

    h_eff_factor = (scaled_period / VIRGO_DISTANCE) * (1 + 0.01 * math.log10(cluster_r + 1))

    entropy_proxy = math.log(disc + 1e-8)
    cohomology_class = entropy_proxy * (rank + 1) / (cluster_r ** 0.5 + 1)

    return {
        'projected_omega': float(scaled_period),
        'h_eff_factor': float(h_eff_factor),
        'comoving_volume': float(comoving_volume),
        'scaled_regulator': float(scaled_reg),
        'entropy_proxy': float(entropy_proxy),
        'cohomology_class': float(cohomology_class),
        'cosmo_scale': float(cosmo_scale),
        'note': 'Full ACSC  $\Phi$  + ECC projection (local DB, high descent)'
    }

```

```

#
=====
=====
# SECTION 4: MAIN EXECUTION WITH 6-CORE PARALLELISM
#
=====
=====

if __name__ == "__main__":
    print("="*120)
    print("  UCF LOCAL DB Tool — Full ACSC Projection")
    print("="*120)

    cluster_data = get_expanded_cluster_data()
    items = list(cluster_data.items())

    results_list = []
    max_workers = 6

    print(f"Starting parallel processing with {max_workers} cores...\n")

    with ProcessPoolExecutor(max_workers=max_workers) as executor:
        future_to_name = {executor.submit(process_cluster, item): item[0] for item in items}

        for future in as_completed(future_to_name):
            try:
                result = future.result()
                results_list.append(result)
                print(f"  ✓ Completed {result['Cluster']}")
            except Exception as exc:
                name = future_to_name[future]
                print(f"  ✗ Error processing {name}: {exc}")
                results_list.append({'Cluster': name, 'Status': f'Parallel Error: {exc}'})

    # Sort results back to original order
    order = list(cluster_data.keys())
    results_list.sort(key=lambda x: order.index(x['Cluster']) if x['Cluster'] in order else 999)

    results_df = pd.DataFrame(results_list)

    print("\n" + "="*120)
    print("          FINAL RESULTS WITH FULL PROJECTION")
    print("="*120)
    print(results_df.to_string(index=False))

```

```
results_df.to_csv('ucf_local_high_descent_acsc_projection.csv', index=False)
print("\n Results saved to 'ucf_local_high_descent_acsc_projection.csv'")
print("Execution complete. Ready for STAR.ipynb import.")
```